124A3013
SE IT

Experiment No 7

Aim:To create a multi-page React application using forms, event handling, routing, refs, and keys.

Requirements :
Node.js + npm
React app created using create-react-app
React Router
npm install react-router-dom

Theory :
Routing enables navigation between pages without reloading (SPA).
Forms + Events handle user input and actions like submit/change.
Refs (useRef) access DOM elements directly (e.g., input value).
Keys uniquely identify list items for efficient rendering.

Code:
src/App.js

```
import React from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Register from "./pages/Register";
import StudentList from "./pages/StudentList";
export default function App() {
return (
<BrowserRouter>
<nav style={{ padding: 12, borderBottom: "1px solid #ccc" }}>
<Link to="/" style={{ marginRight: 12 }}>Register</Link>
<Link to="/list">Student List</Link>
</nav>
<div style={{ padding: 16 }}>
<Routes>
<Route path="/" element={<Register />} />
<Route path="/list" element={<StudentList />} />
</Routes>
</div>
</BrowserRouter>
);
}
```

src/pages/Register.js

```
import React, { useRef, useState } from "react";
```

```
export default function Register() {
const nameRef = useRef(null);

const deptRef = useRef(null);
const [message, setMessage] = useState("");
const handleSubmit = (e) => {
e.preventDefault(); // event handling
const name = nameRef.current.value.trim();
const dept = deptRef.current.value.trim();
if (!name || !dept) {
setMessage("Please enter both Name and Department.");
return;
}
// Save in localStorage (simple way to share data between pages)
const existing = JSON.parse(localStorage.getItem("students") || "[]");
existing.push({ id: Date.now(), name, dept });
localStorage.setItem("students", JSON.stringify(existing));
setMessage("Student registered successfully!");
e.target.reset();
nameRef.current.focus();
};
return (
<div style={{ maxWidth: 420 }}>
<h2>Student Registration</h2>
<form onSubmit={handleSubmit}>
<label>Name</label>
<input
ref={nameRef}
type="text"
placeholder="Enter name"
style={{ width: "100%", padding: 10, margin: "6px 0 12px" }}
/>
<label>Department</label>
<input
ref={deptRef}
type="text"
placeholder="Enter department"
style={{ width: "100%", padding: 10, margin: "6px 0 12px" }}
/>
<button type="submit">Submit</button>
</form>
<p style={{ marginTop: 12 }}>{message}</p>
</div>
);
```

```
}
```

src/pages/StudentList.js

```
import React, { useEffect, useState } from "react";
export default function StudentList() {
const [students, setStudents] = useState([]);
useEffect(() => {
const data = JSON.parse(localStorage.getItem("students") || "[]");
setStudents(data);

}, []);
return (
<div>
<h2>Student List</h2>
{students.length === 0 ? (
<p>No students found. Please register first.</p>
) : (
<ul>
{students.map((s) => (
<li key={s.id}>
{s.name} - {s.dept}
</li>
))}
</ul>
)}
</div>
);
}
```

Output:

Register StudentList

Student Registration

Name

Dakshata

Department

IT

Submit

Student Registration

Name

Enter name

Department

Enter department

Submit

Student registered successfully!

Register StudentList

Student List

- Namrat - IT
- Namrata - IT
- Dakshata - IT

Conclusion:

Building a multi-page React application integrates **React** seamless navigation and **Ref hooks** to manage direct DOM interactions or persistent values without re-renders. By implementing controlled components for **form handling** and utilizing unique **keys** for efficient list rendering, the app ensures high performance and data integrity. This combination of tools allows for a scalable, interactive user experience with organized state management and smooth transitions between views